

System Scanner Pro

AI-Powered Distributed Endpoint Monitoring & Remote Scanning Platform

Technical Documentation Package | v3.1

Installation & Configuration Guide

Verified step-by-step setup for local, VPS and serverless deployments

Project	System Scanner Pro (v3.1)
Document	Installation & Configuration Guide
Package date	August 21, 2026
Prepared by	Project Team - Kailasanathan (Team Leader), Saagr, Mega
Audience	Developers, administrators, testers and new team members
Status	Final

This document is part of a nine-volume documentation package generated from the actual project source code. All commands, file paths, endpoints and behaviours described here were verified against the repository at documentation time.

Table of Contents

1. About This Guide	3
2. Prerequisites Checklist	3
3. Getting the Code	3
4. Understanding Environment Variables	3
5. Creating Your .env File	4
6. Local Installation - Server (Windows/macOS/Linux)	4
7. First Run - Admin Server	4
8. Verifying the Server Is Up	4
9. Default Login & First Actions	4
10. Running the Client Agent from Source	5
11. Approving Your First Client	5
12. Building the Windows Executable	5
13. Agent Autostart Behaviour	5
14. Database Options Explained	5
15. Provisioning Supabase (Cloud Mode)	5
16. Cloud Discovery Flow	6
17. VPS Deployment - One-Command Install	6
18. VPS Deployment - Manual Steps	6
19. Firewall & Ports Reference	6
20. Deploying to Vercel (Serverless)	6
21. Static Files & WhiteNoise	7
22. Upgrading an Existing Deployment	7
23. Backup Procedures	7
24. Restore Drill	7
25. Uninstall / Cleanup	7
26. Troubleshooting - Server Won't Start	7
27. Troubleshooting - Agents Not Appearing	8
28. Troubleshooting - Real-Time Features Silent	8
29. Performance Tuning Quick Wins	8
30. Security Hardening Checklist (Deployment)	8
31. Verification Checklist After Install	8
32. Quick Reference Card	9
33. Where to Go Next	9

1. About This Guide

Every command below was verified against the repository. Where a value must be supplied by you (passwords, domains, keys), it is marked [TO BE PROVIDED]. This guide never reproduces real credentials; the tracked .env.template in the repo contains live-looking values and should be treated as compromised - see section 32.

2. Prerequisites Checklist

- Python 3.10 or newer on the server host (Vercel uses 3.11 via runtime.txt).
- pip and venv available.
- Git client to clone <https://github.com/shinecraftinternship-kms/AdminClient.git>
- For VPS: Debian/Ubuntu with sudo, ports 80/443 open, a DNS A record pointing at the host.
- For cloud mode: a Supabase project URL and service key [PROVIDED SEPARATELY - never commit].
- Client machines: none for the packaged exe; Python 3.10+ if running from source.

3. Getting the Code

```
git clone https://github.com/shinecraftinternship-kms/AdminClient.git
cd AdminClient
```

Repository layout highlights: admin/ (Django server), client/ (agent source), api/index.py (serverless entry), deploy/ (nginx, systemd, install.sh), docs/, setup_new_supabase.sql, setup_cloud_discovery.sql.

4. Understanding Environment Variables

Configuration is environment-driven. The table lists every variable the code reads and its effect. Values are yours to supply; nothing below is a real secret.

Variable	Used by	Purpose / default behaviour
DJANGO_SECRET_KEY	settings.py	Signing key for sessions/cookies/JWT. If unset, an insecure development fallback is used - ALWAYS set this in production.
DJANGO_DEBUG	settings.py	'False' disables debug. Defaults to True when unset - set False in production.
DJANGO_ALLOWED_HOSTS	settings.py	Comma-separated hosts. Defaults to ['*'] - restrict in production.
DATABASE_URL	settings.py via dj-database-url	PostgreSQL connection string. On serverless the code forces the Supabase pooler transaction port (6543) and sslmode=require.
DB_CONN_MAX_AGE	settings.py	Persistent-connection seconds; forced to 0 on serverless.
SUPABASE_URL	scanner_api/supabase_client.py	Supabase project REST base for discovery registry.
SUPABASE_SERVICE_KEY	scanner_api/supabase_client.py	Service-role key used ONLY server-side to publish/read the server_registry row. [PROVIDED SEPARATELY]
RESET_TOKEN	api/index.py	Guards the POST /__reset/ maintenance endpoint. A hardcoded fallback exists in code - override it in any shared deployment.
ADMIN_SERVER_URL	.github/workflows + agent	Public base URL published to server_registry by the hourly keep-alive Action.

Table 1 - Environment variables

5. Creating Your .env File

```
# admin/.env (never commit this file)
DJANGO_SECRET_KEY=[TO BE PROVIDED - e.g. 50+ random chars]
DJANGO_DEBUG=False
DJANGO_ALLOWED_HOSTS=your-domain.example.com
# local SQLite mode: omit DATABASE_URL entirely
# cloud mode:
# DATABASE_URL=postgresql://<user>:<password>@<host>:5432/postgres
# SUPABASE_URL=https://<project-ref>.supabase.co
# SUPABASE_SERVICE_KEY=[PROVIDED SEPARATELY]
```

Security notice

The repository currently tracks a file named **.env.template** that contains **real-looking credentials** (database password, service key, JWT secret). Do not copy it as-is. Rotate those secrets and replace the template with placeholders - see volume 06 known issues.

6. Local Installation - Server (Windows/macOS/Linux)

1. python -m venv .venv then activate it (Windows: .venv\Scripts\activate ; POSIX: source .venv/bin/activate)
2. pip install -r requirements.txt
3. Copy the template above into admin/.env and fill values.
4. python admin/manage.py migrate
5. python admin/manage.py createsuperuser # optional; main.py auto-seeds admin otherwise

7. First Run - Admin Server

```
# from repository root
python admin/main.py
# options: --port 8000 --host 0.0.0.0 --asgi --domain your.host
#          --signed-cookies --no-open
```

main.py applies pending migrations, seeds a default superuser when none exists, starts the APScheduler background jobs, registers the ADMIN self-client, launches a UDP discovery listener on port 45000, and serves HTTP on the chosen port (default 80; use --port 8000 for unprivileged runs). With --asgi it runs Daphne so WebSockets work.

8. Verifying the Server Is Up

- Open <http://localhost:8000/> - login page appears.
- GET <http://localhost:8000/api/health/> returns JSON status.
- Log output appears in console and admin/local_server.log.

9. Default Login & First Actions

Change immediately

When no superuser exists, main.py seeds username **admin** with a default password (documented in README) and prints it once. Log in, open Settings and change it before enrolling any device.

1. Browse to <http://localhost:8000/login/> and sign in.
2. Open Settings: change password, review company name and scan defaults.
3. Note the Connect URL shown on the dashboard - agents can parse it directly.

10. Running the Client Agent from Source

```
# on the managed machine
pip install -r client/requirements.txt
python client/main.py "http://<server-host>:8000/connect/<username>/<company-slug>/"
```

Without arguments the agent attempts UDP discovery on port 45000, then falls back to the Supabase registry when configured. Useful flags include `--once` (single registration/scan cycle), `--rescue` (diagnostics dump), `--silent` (no console window interactions) and `--uninstall` (removes autostart entries). Run `python client/main.py --help` for the full list.

11. Approving Your First Client

1. Dashboard shows the new machine under Pending Clients with hostname, platform and fingerprint.
2. Click Approve. The device flips to online within one heartbeat (30 s).
3. Trigger Scan Now and open the device detail page to inspect inventory.

Approval rules enforced by code and unit tests: a different fingerprint under the same key resets approval; ping from a never-approved device keeps it Pending; explicit admin approval survives pings; approving a soft-deleted client reactivates it.

12. Building the Windows Executable

```
pip install pyinstaller # 6.x
cd "C:\...\AdminClient"
python build_client.py
# output: dist/client_scanner.exe (+ zipped copy)
```

`build_client.py` wraps `client/main.py` with PyInstaller, embedding version metadata and an icon. Distribute `client_scanner.exe` (or the zip) to machines; no Python is required there.

13. Agent Autostart Behaviour

- Windows: writes a Registry Run key and/or Startup-folder launcher pointing at the exe.
- Single-instance mutex prevents duplicate agents per machine.
- Watchdog thread restarts the heartbeat loop if it dies; offline queue preserves events.

14. Database Options Explained

Mode	When	Setup
SQLite (default)	Local demo or single-node VPS	Nothing - file lives in <code>admin/data/scanner.db</code> (auto-migrated to a persistent OS data dir when frozen; covered by unit test).
PostgreSQL via DATABASE_URL	Multi-process or cloud	Set <code>DATABASE_URL</code> ; run migrate against it.
Supabase hosted Postgres	Serverless/cloud	Run <code>setup_new_supabase.sql</code> in the Supabase SQL editor; set <code>DATABASE_URL</code> to the pooler URI (6543).

Table 2 - Choosing a database mode

15. Provisioning Supabase (Cloud Mode)

1. Create a project at supabase.com; note the project ref and region.
2. SQL Editor -> run `setup_new_supabase.sql` (creates clients, scan_results, addon_devices, server_registry and related tables/indexes).
3. Optionally run `setup_cloud_discovery.sql` to reset the discovery row.

4. Project Settings -> API: copy the project URL and service-role key into your .env [PROVIDED SEPARATELY].

16. Cloud Discovery Flow

Agents try, in order: direct connect URL argument, UDP broadcast on 45000, then the Supabase server_registry row. The admin server publishes its own public URL into that table at startup; a GitHub Actions workflow (.github/workflows/supabase-register.yml) re-publishes ADMIN_SERVER_URL hourly using repository variables/secrets as a safety net.

17. VPS Deployment - One-Command Install

```
sudo DOMAIN=your-domain.example.com bash deploy/install.sh
```

The script targets Debian/Ubuntu and performs: package installs (nginx, certbot), user and directory creation (/opt/scanner-admin), venv + requirements, migrations, systemd unit installation (system-scanner-admin.service running admin/main.py --asgi --domain), nginx site copy from deploy/nginx/scanner.conf, and Certbot certificate issuance.

18. VPS Deployment - Manual Steps

1. Clone repo to /opt/scanner-admin; create venv; pip install -r requirements.txt.
2. Write /opt/scanner-admin/admin/.env with production values.
3. cp deploy/systemd/system-scanner-admin.service /etc/systemd/system/ ; edit paths/user if needed.
4. systemctl daemon-reload && systemctl enable --now system-scanner-admin
5. cp deploy/nginx/scanner.conf /etc/nginx/sites-available/scanner ; symlink into sites-enabled;
6. Edit server_name; nginx -t && systemctl reload nginx.
7. certbot --nginx -d your-domain.example.com

19. Firewall & Ports Reference

Port	Protocol	Direction	Purpose
80 / 443	TCP	inbound (public)	HTTP redirect + HTTPS dashboard/API (nginx)
8000	TCP	localhost only	Daphne/upstream behind nginx
45000	UDP	LAN broadcast	Agent discovery ('DISCOVER_ADMIN' probe/reply)
5432 / 6543	TCP	outbound	Postgres direct / Supabase pooler transaction mode
443	TCP	outbound from agents	WSS + HTTPS to server

Table 3 - Port matrix

20. Deploying to Vercel (Serverless)

1. Push the branch to GitHub; import the project in Vercel (framework preset: Other).
2. vercel.json already maps all routes to api/index.py with function maxDuration 30 s.
3. Set environment variables in the Vercel dashboard: DJANGO_SECRET_KEY, DJANGO_DEBUG=False, DJANGO_ALLOWED_HOSTS=.vercel.app (+ custom domain), DATABASE_URL (Supabase pooler URI), optionally SUPABASE_URL/SUPABASE_SERVICE_KEY.
4. Deploy. First request runs migrations-safe bootstrap via api/index.py (whitenoise static, signed-cookie sessions forced).
5. Set repository variable ADMIN_SERVER_URL to the deployment URL so the keep-alive Action publishes it for agent discovery.

Serverless capability note

WebSockets require ASGI; the Vercel path serves REST endpoints and pages. Real-time push features are designed for the VPS/Daphne deployment.

21. Static Files & WhiteNoise

collectstatic output is served by WhiteNoise in both deployment modes; no separate static server is needed. CDN assets (Bootstrap/Chart.js) load from public CDNs at page render time.

22. Upgrading an Existing Deployment

1. git pull (or upload new release).
2. Activate venv; pip install -r requirements.txt.
3. python admin/manage.py migrate.
4. Restart service: systemctl restart system-scanner-admin (or redeploy on Vercel).
5. Rebuild/redistribute client_scanner.exe if client code changed (version string prints at agent start).

23. Backup Procedures

- SQLite: stop service, copy admin/data/scanner.db (plus WAL sidecar files if present) to safe storage; restart.
- PostgreSQL: pg_dump "\$DATABASE_URL" > backup_\$(date +%F).sql ; restore with psql < backup.sql.
- Secrets: back up .env separately and encrypted - it is intentionally not in git.

24. Restore Drill

1. Provision a clean host (or existing host with service stopped).
2. Restore database file/dump to the expected location.
3. Start service; verify /api/health/ and dashboard counts match pre-backup snapshot.

25. Uninstall / Cleanup

```
# agent machine
python client/main.py --uninstall
# server host (VPS)
sudo systemctl disable --now system-scanner-admin
sudo rm /etc/nginx/sites-enabled/scanner /etc/nginx/sites-available/scanner
sudo certbot delete --cert-name your-domain.example.com
```

Removing the repo directory deletes code; delete the database file/volume separately if you also want data gone.

26. Troubleshooting - Server Won't Start

Symptom	Likely cause	Fix
Port already in use	Another process on 80/8000	Use --port 8000 or stop the conflicting service
Migration errors about relations	DB points at empty/wrong schema	Run manage.py migrate; verify DATABASE_URL
psycopg2 SSL errors	Direct 5432 blocked	Use pooler URI (6543) with sslmode=require
401 on every API call after restart	SECRET_KEY changed	Keep SECRET_KEY stable; re-login clients
Static files 404 on Vercel	collectstatic skipped	Ensure build runs api/index.py bootstrap; check whitenoise config

Table 4 - Server startup issues

27. Troubleshooting - Agents Not Appearing

- Wrong key/URL: re-run agent with the exact connect URL from the dashboard.
- UDP blocked: discovery falls through to registry/URL modes; check firewall on 45000 for LAN discovery.
- Clock skew: HMAC timestamps tolerate +/-300 s; fix system time if signatures fail.
- Antivirus quarantine of unsigned exe: allowlist client_scanner.exe or run from source.
- Already registered: same fingerprint re-uses identity; a changed fingerprint intentionally demotes to Pending.

28. Troubleshooting - Real-Time Features Silent

- Confirm server runs with --asgi (Daphne); WSGI-only serving has no WebSocket upgrade.
- nginx must forward Upgrade/Connection headers (provided config does).
- On Vercel, expect REST-only behaviour by design.

29. Performance Tuning Quick Wins

- Raise DB_CONN_MAX_AGE on persistent servers (e.g. 60) to reuse connections.
- Increase scan_interval for stable machines to reduce load.
- Run PostgreSQL instead of SQLite beyond ~50 active devices.
- Give the VM 2 vCPU when anomaly/predictive checks run across many devices.

30. Security Hardening Checklist (Deployment)

1. DJANGO_SECRET_KEY set to a unique random value; DJANGO_DEBUG=False; ALLOWED_HOSTS restricted.
2. Default admin password changed; invite real users via signup with strong passwords.
3. RESET_TOKEN overridden from its hardcoded fallback on any shared host.
4. TLS enabled (Certbot) and HTTP redirected to HTTPS (nginx config does this).
5. Rotate/remove the credentials committed in .env.template; scrub git history if possible.
6. Restrict CORS (settings currently allow all origins) if the API need not be public.

31. Verification Checklist After Install

Check	Expected result
Login page loads over HTTPS	200 OK, dark-themed UI
/api/health/	JSON success payload
Agent enrolled	Pending card appears within seconds
After approval	Device online within ~30 s; heartbeat visible
Scan Now	Scan history entry created; inventory populated
WebSocket	Live status changes without manual refresh (VPS mode)
Report export	PDF/Excel/CSV downloads open correctly

Table 5 - Post-install smoke test

32. Quick Reference Card

SERVER

```
python admin/main.py [--port N] [--asgi] [--domain H] [--signed-cookies]
python admin/manage.py migrate | createsuperuser | scan_all | clear_data
```

AGENT

```
python client/main.py "<connect-url>" [--once] [--silent] [--rescue] [--uninstall]
```

BUILD

```
python build_client.py -> dist/client_scanner.exe
```

VPS

```
sudo DOMAIN=host bash deploy/install.sh
```

SERVICE

```
systemctl status|restart system-scanner-admin ; journalctl -u system-scanner-admin -f
```

33. Where to Go Next

Volume 04 explains the codebase file-by-file; volume 07 walks administrators through daily UI operations; volume 08 covers testing and maintenance routines.